

# **ASSIGNMENT – 4**

**CSA1006**

## **Software Engineering**

**Name:** P.Poojitha

**Register**

**No:**192472349

**1. Describe the Key Stages of a CI/CD Pipeline (using Jenkins or GitHub Actions). How does using Docker for containerization support continuous delivery?**

**Ans:**

### **CI/CD Pipeline**

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment.

- Continuous Integration (CI): Developers frequently integrate their code into a shared repository, where automated builds and tests are performed.
- Continuous Delivery (CD): Validated software is automatically prepared for release and can be deployed quickly.

### **Key Stages of a CI/CD Pipeline**

Code → Build → Test → Package → Deploy → Monitor

#### **1. Source Code Management**

Developers write code and push it to a repository such as GitHub.

Example:

Developer → Git → GitHub

A push or pull request can automatically trigger the pipeline.

#### **2. Build**

The CI tool, such as Jenkins or GitHub Actions, downloads dependencies and compiles/builds the application.

Example:

Source Code → Dependencies → Build

Build errors are reported immediately.

### **3. Automated Testing**

Automated tests are executed to identify errors.

Common tests include:

- Unit testing
- Integration testing
- Functional testing
- Security testing

If tests fail, the pipeline can stop automatically.

### **4. Package**

After successful testing, the application is packaged into a deployable format.

### **5. Deployment**

The application is deployed to a testing, staging, or production environment.

Deployment can be performed automatically after all required checks pass.

### **6. Monitoring and Feedback**

After deployment, the application is monitored for:

- Errors
- Performance
- Availability
- Resource usage

Feedback can be used for further development.

## **How Docker Supports Continuous Delivery**

Docker packages an application together with its required libraries, dependencies, and configuration into a container.

### **Advantages of Docker in CI/CD**

1. Consistent environment: The application runs in the same environment during development, testing, and production.
2. Dependency management: Required libraries and dependencies are packaged with the application.
3. Portability: Containers can run across different systems and cloud platforms.
4. Faster deployment: Containers can be created and started quickly.
5. Isolation: Applications run in isolated environments.
6. Easy rollback: Previous container images can be redeployed if a new version has problems.
7. Scalability: Multiple containers can be created when application demand increases.

A CI/CD pipeline automates the process from code integration to testing and deployment. Docker strengthens continuous delivery by providing a consistent, portable, and reproducible environment, making software releases faster and more reliable.

← CI/CD Pipeline

## ✓ Update app.js #12

Re-run all jobs



### Summary

#### Jobs

- ✓ Checkout Code 2s
- ✓ Set up Node.js 3s
- ✓ Install Dependencies 5s
- ✓ Run Tests 7s
- ✓ Build Project 4s
- ✓ Build Docker Image 6s
- ✓ Push Docker Image 8s
- ✓ Deploy to Server 5s

#### Run details

- Usage
- Workflow file

Triggered via push 1 minute ago

Status

Total duration

Artifacts

VarshaReddy pushed ← 8f3c2d1 main

Success

40s

1

### Run Tests

Search logs



#### Run npm test

7s

```
1 > ci-cd-demo@1.0.0 test
2 > jest --coverage
3
4 PASS tests/app.test.js
5   ✓ addition works (3 ms)
7   ✓ subtraction works (1 ms)
8   ✓ multiplication works (1 ms)
9   ✓ division works (1 ms)
```

| File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|-----------|---------|----------|---------|---------|-------------------|
| All files | 100     | 100      | 100     | 100     |                   |
| app.js    | 100     | 100      | 100     | 100     |                   |

```
15 Test Suites: 1 passed, 1 total
17 Tests:      4 passed, 4 total
18 Snapshots:  0 total
20 Time:       1.123 s
21 Ran all test suites.
```